Mid-Semester Progress Report

Summer 2026– DSA 5900– 1 Credit Hour

Danny Nguyen

Faculty Supervisor: Dr. Matthew Beattie
Thesis Advisor: Dr. Charles Nicholson

Contents

List of Figures

1 Introduction

Translating natural language into formal predicate logic is a fundamental prerequisite for automated reasoning, yet the inherent ambiguity of human language makes this process notoriously difficult for current generative pipelines. When tasked with outputting structured logical configurations, generative LLMs often succumb to hallucination and parsing fragility, rendering the resulting logical structures unreliable [3, 4, 5, 8].

This practicum proposes a shift in methodology: bypassing text-generation in favor of parameter-level probing. Rather than forcing models to generate structured output, we embrace the model's existing internal representations. The central objective is to explore the model's internal geometry to reveal whether logical relationships such as: implication, negation, and conjunction; are explicitly encoded within the hidden state vector embeddings of Meta's Llama 3.2-3B [1, 6]. By analyzing these internal activation patterns, this practicum aims to determine if verifiable predicate logic can be extracted deterministically from the latent space of a compact transformer, offering a robust, low-compute alternative to stochastic generative parsing.

An example of the training and subsequent inference behavior of a negation detection probe follows:

- **Input:**
  - Natural Language Text Statement: "Eli is not soft."
  - Atomic Elements: ["Eli", "soft"]
- **Output:** Log-Odds Score of Negation Presence between the Atomic Elements

likewise for an implication detection probe:

- **Input:**
  - Natural Language Text Statement: "If someone is soft, he is poised and not scared."
  - Atomic Elements: ["soft", "poised", "not scared"]
- **Output:** Log-Odds Score of Implication Presence between the Atomic Elements

With the same principle mirrored for training classification probes for other desired logical relationships such as equivalence, conjunction, and disjunction.

A nominal example of a pipeline utilizing successfully trained probes is proposed in Figure 1 with the corresponding visualization of the resulting graph in Figure 2.

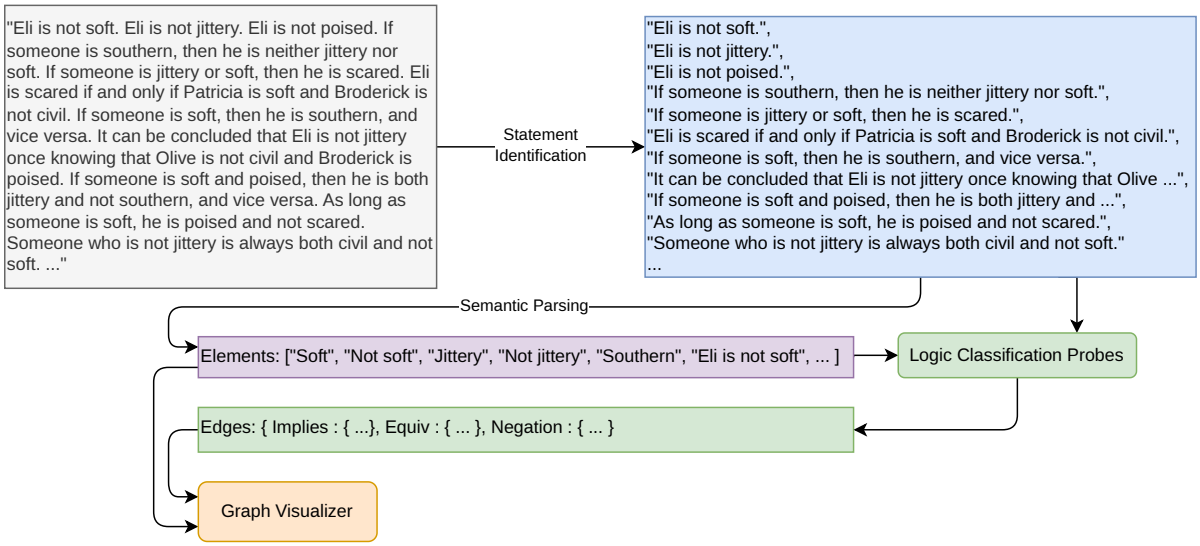

Figure 1: Pipeline from natural language to logical graph representation.

2 Objectives

The overarching goal of this practicum is to develop a robust, parameter-level extraction pipeline that bypasses generative LLM bottlenecks to map natural language into verifiable logical graphs. The specific project and individual objectives are outlined below:

2.1 Technical Project Objectives

- **Latent Space Probing Architecture:** Design and train low-compute, linear classification heads (probes) to extract propositional claims and logical relationships directly from the intermediate residual stream of Meta's Llama 3.2-3B model.
- **Symbolic Pipeline Integration:** Construct a deterministic pipeline that translates extracted latent representations into discrete symbolic tokens without relying on stochastic text-generation or JSON parsing.
- **Graph-Based Fact Verification:** Implement a visualization and verification layer that structures the extracted symbols into a formal logical graph, for human interpretation of the underlying logical structures of natural language.
- **Performance Optimization:** Evaluate the classifier framework to achieve high factual and structural precision while reducing inference-time computational overhead compared to base-

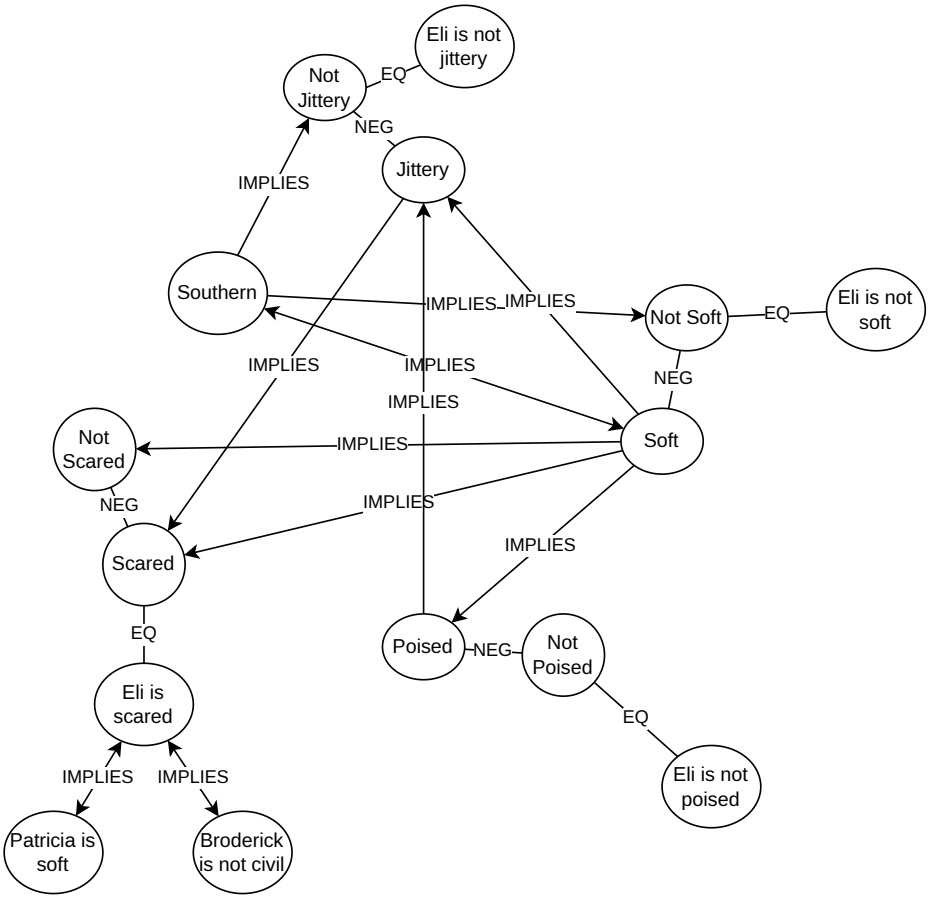

Figure 2: Visualization of the logical graph extracted from the natural language example.

line autoregressive generative prompting.

2.2 Individual Learning Objectives

- **Mechanistic Interpretability Mastery:** Gain deep, practical expertise in analyzing the internal geometry of transformers, specifically concerning layer-wise activation tracking, representation dynamics, and internal knowledge localization.
- **Advanced Representation Engineering:** Master state-of-the-art diagnostic probing and activation patching methodologies using specialized interpretability toolkits.
- **Symbolic Architecture Development:** Develop a rigorous engineering workflow for bridg-

ing machine representations (vector embeddings) with classical symbolic paradigms (graph structures).

3 Data

3.1 Ingestion

The dataset selected for this project is LogicNLI, an open-source diagnostic benchmark hosted on the Hugging Face hub and developed by Tian et al.. LogicNLI was specifically created to address growing concerns regarding whether modern language models (LMs) truly possess reasoning capabilities or if they merely rely on spurious statistical correlations.

As a Natural Language Inference (NLI)-style dataset, LogicNLI is designed to isolate First-Order Logical (FOL) reasoning from "common sense" and domain knowledge. It accomplishes this by utilizing a synthetic composition consisting of triplets of "facts", "rules", and "statements" that test a model's ability to handle fundamental logical operators; such as conjunction, disjunction, negation, implication, equivalence, and universal and existential quantifiers. This synthetic approach is highly beneficial for this study because it provides a controlled environment to evaluate model performance across four key diagnostic perspectives: overall accuracy, robustness to irrelevant information, more-hop generalization, and proof-based traceability[7].

The data is publicly available and well-documented. For this project the dataset is stored locally in a structured directory within the project repository, ensuring easy access for preprocessing and analysis.

3.2 Exploration

Initial exploratory data analysis revealed a highly uniform, balanced, and synthetically controlled dataset structure across its 20,000 entries. The target `labels` field contains a near-perfect balanced split across its classification categories (Entailment, Contradiction, Self-Contradiction, and Neutral) eliminating the risk of majority-class bias during probe calibration. A structural analysis of the text fields highlights the specific syntactic attributes of the dataset:

- **Facts Array:** This field contains flat, atomic assertions of absolute truth (e.g., `["Alice is in the garden.", "Bob is hungry."]`). These strings exhibit zero logical connectives, maintaining a short average string length of 4 to 6 tokens.
- **Rules Array:** This field holds the propositional conditional formulas (e.g., `["If Alice is in the garden, then Alice is reading."]` or structures introducing logical equivalence and conjunction (`"and"`)). These strings demonstrate higher token complexities and specific syntactic anchors that dictate the relationship edges.
- **Statements and Labels:** The `statements` field presents the test query, which correlates cleanly to an explicit truth value defined by the formal deduction of the combined rules and

facts.

An abbreviated example of one raw record from LogicNLI's dataset is shown in Figure 3.

```
{
  "facts": [
    "Eli is not soft.", "Patricia is civil.", "Broderick is soft.",
    "Paul is civil.", "Miles is not southern.", "Paul is not scared.",
    "Ronald is jittery.", "Broderick is not scared.",
    "Broderick is not poised.", "Paul is not poised.",
    "Eli is not jittery.", "Eli is not poised."
  ],
  "rules": [
    "If someone is southern, then he is neither jittery nor soft.",
    "If someone is jittery or soft, then he is scared.",
    "As long as someone is soft, he is poised and not scared.",
    "Someone who is not jittery is always both civil and not soft."
    ...
  ],
  "statements": ["Patricia is scared.", "Eli is not soft."],
  "labels": ["entailment", "entailment"]
}
```

Figure 3: Example data record from the LogicNLI dataset, illustrating the synthetic structure of facts, rules, and logical statements.

Exploratory token length histograms further confirmed tight distribution peaks, ensuring that the model's intermediate hidden layers will not suffer from sequence padding distortions during batch processing.

3.3 Preparation

Because the linear probes require distinct feature-target pairs to map out Llama's internal vector space, substantial preprocessing and feature engineering were executed to transform the raw dataset into a token-level sequence classification task. No missing data or null fields were detected due to the synthetic nature of LogicNLI, bypassing the need for standard statistical imputation. The data preparation workflow consists of two critical pipeline stages:

1. **Rule-Based Target Transformation and Fact-Rule Matching:** A custom semantic parsing script was built to map the structural elements of the environment. Within the `rules`

array, regular expressions and token-dependency parsing isolate the relational connectives, explicitly identifying whether a rule represents an implication ($\rightarrow$) or a conjunction ($\wedge$). Concurrently, a lookup matrix maps these rules directly back to the atomic strings inside the `facts` array. This enables the framework to identify exactly which baseline facts are being operated on by any given rule, providing clear structural targets for the relational edge probes.

2. **Contradiction-Driven Negation Curation:** Because explicit negation ($\neg$) is frequently represented implicitly across text pairs rather than through simple keyword matching, a target-generation strategy was implemented leveraging the dataset's logical symmetries. The pipeline filters entries where the target `label` is explicitly marked as a `"contradiction"`. By aligning the queried `statement` against its corresponding asset in the `facts` array under this specific label, the pipeline mathematically isolates the exact token span undergoing negation. This contradiction-driven mapping provides a high-precision, curated subset of explicit positive-to-negative vector transformations.

3. **Token Span Realignment:** Because character indices shift once strings are converted into transformer sub-tokens, the Hugging Face tokenizer was configured with the `return_offsets_mapping=True` parameter. This maps the character spans generated during the rule-parsing and contradiction-alignment steps directly onto the sequence tensor positions of Meta's Llama 3.2-3B model. This preparation ensures that when forward hooks intercept the hidden states of layers 8 through 16, the mean-pooling functions grab the precise vector slices corresponding to the isolated logical components.

4 Methodology

4.1 Techniques

To extract and verify the logical structures embedded within Meta's Llama 3.2-3B model, this practicum utilizes a mechanistic interpretability framework centered on diagnostic linear "probing".

Transformers In the standard Transformer architecture, input sequences are processed through a stack of L sequential transformer blocks. For any given token at position i and layer l, the model generates a hidden state activation vector $h_i^{(l)} \in \mathbb{R}^d$. This vector serves as the distributed, high-dimensional representation of the token's contextual state, encoding the compositional hierarchy of syntactic dependencies and semantic features derived from the preceding attention and feed-forward operations.

These activations persist within the "residual stream", an architectural design that facilitates the additive accumulation of information across the network's depth. In this framework, each transformer block l performs an incremental update $h_i^{(l+1)} = h_i^{(l)} + \text{sublayer}(h_i^{(l)})$, ensuring that the residual stream serves as a continuous conduit for information propagation. This project posits that this latent space encodes propositional logical structures—such as implications, negations, and conjunctive relations—as geometric properties within the manifold of the hidden state embeddings, independent of, and prior to, the final logit projection onto the vocabulary space.

By applying diagnostic linear classification "probes" to these frozen activation vectors, the linear separability of these logical relations can be formally tested. Specifically, if a discrete logical relation $Y \in \{0, 1\}$ is decodable via a weight matrix W, it provides empirical evidence that the information is explicitly formatted in a linearly recoverable configuration within the model's internal representation of the input.

Diagnostic Linear Classification Probing To facilitate the extraction of latent logical structures, this project implements diagnostic linear classification, utilizing the methodology established by Alain and Bengio (2018). The core principle of linear probing involves treating the internal activation vectors of a frozen transformer model as a fixed feature space. By training a simple, low-capacity linear model—a "probe"—to map these high-dimensional hidden state vectors to discrete logical class labels, the linear separability of internal concept representations within the model's latent geometry can be empirically assessed.

In contrast to end-to-end fine-tuning, which modifies the parameters of the base architecture, linear probing relies on the gradient descent optimization of a shallow classifier using frozen activation outputs. High classification accuracy on held-out test sets provides empirical evidence that the

target logical relations are explicitly formatted in a linearly recoverable configuration at the probed layer. Conversely, insufficient classification performance suggests that the information is either absent from the latent representation or encoded within a non-linear manifold that necessitates a more complex, high-capacity decodable mapping.

4.1.1 Algorithmic Formulation and Modeling

To quantify the presence of logical relations within the transformer's latent space, this project employs shallow linear classification probes. A "hidden state activation vector" is the high-dimensional numerical output of a transformer block at a specific layer, representing the model's contextual understanding of an input token. A "shallow linear probe" is a low-capacity classifier (specifically, logistic regression) designed to identify patterns in these vectors without modifying the underlying transformer parameters. The "discrete logical class Y" refers to the target category—such as the presence of an implication or a negation—that the probe is trained to predict from the hidden state activation vector.

The classification process is optimized via Logistic Regression with L_2 regularization to prevent over-fitting to the training samples. Given a dataset of hidden state activation vectors $X \in \mathbb{R}^d$ extracted from a specific intermediate transformer layer l (where $d = 2048$ for Llama 3.2-3B), the probe learns a weight matrix W and bias vector b to map the input hidden state X to the discrete logical class Y:

$$\hat{Y} = \sigma(W \cdot X + b)$$

where σ denotes the sigmoid function. The weight matrix W acts as a learned linear decision boundary within the d-dimensional embedding space. By optimizing W via gradient descent, the probe identifies the hyperplane that optimally separates the hidden state vectors corresponding to the presence versus the absence of a logical relation. The output $\hat{Y}$ represents the predicted probability of the logical class. This mapping is derived from the linear log-odds (logit), where $W \cdot X + b$ defines the decision boundary in the latent space.

Once trained, W serves as a diagnostic tool for logical decoding. During evaluation, new hidden state vectors are projected through the learned W and b; the resulting scalar output $\hat{Y}$ indicates the model's internal assignment of a logical class to that specific token position. High classification accuracy on unseen test data demonstrates that the logical relation Y is not merely implicit but is explicitly organized as a linearly separable feature in the model's internal representation, thereby validating the model's capacity to internalize structured logical operations. Conversely, poor classification performance suggests the absence of a linearly recoverable representation of the target logical relation.

4.1.2 Train-Test Splits and Validation Strategy

The experimental validation utilizes a predefined train/dev/test split strategy, leveraging the existing partitions within the LogicNLI benchmark to ensure a rigorous evaluation of the latent logical representations. The dataset comprises a total of 20,000 samples distributed as 8,000 for training, 1,000 for development (dev), and 1,000 for testing. To minimize bias, class distributions across all splits are balanced, ensuring that the probes are trained on an equal representation of "entailment" and "contradiction" (or the presence versus absence of specific logical relations).

Individual probes are instantiated for each target logical category to isolate their distinct representation within the latent space. Separate models are trained for the following primitives:

- **Propositional Connectives:** Implication ($\rightarrow$), Negation ($\neg$), and Conjunction ($\wedge$).
- **Atomic Terms:** Predicate and entity extraction (e.g., *Studies(x)*, *Passes(x)*).

The validation strategy adheres to the following pipeline:

- **Calibration Split:** Probes are trained using activation vectors extracted strictly from the 8,000 records of the `train` split. Baseline performance is established by comparing the linear probe results against a majority-class classifier, ensuring that any successful extraction is indicative of meaningful latent organization rather than heuristic guessing.
- **Model Selection and Layer Auditing (Dev Split):** The validation framework utilizes the `dev` split to audit the model layer-by-layer. Probes are independently trained and evaluated across layers 8 through 16 to map the trajectory of logical synthesis. This specific layer range is selected based on empirical precedents in transformer interpretability, which suggest that mid-to-late layers frequently encode the most sophisticated contextual and relational information prior to the final output layers [6]. The layer yielding the highest F1-score on the dev set is designated as the optimal extraction layer for that specific primitive.
- **Generalization Verification (Test Split):** Final model performance is reported strictly using vectors from the unseen `test` split. Out-of-sample generalization is quantified using Precision, Recall, and the F_1-score relative to the ground-truth target structures, providing a robust measurement of whether the model has truly internalized the logical relationship as a generalizable feature.

4.2 Process Validation

The choice to implement a parameter level probing methodology rather than standard auto-regressive token generation was refined through consultations with the thesis advisor about the application of modern data science techniques to automated fact checking. These discussions led to delving into

the space of mechanistic interpretability research largely around the problems of these token generation models when applied to logical reasoning tasks. While auto-regressive prediction is the standard paradigm for language modeling, it is often suboptimal for diagnostic tasks. Auto-regressive loops generate output sequentially, compounding potential errors through stochastic sampling; when an LLM is forced to output structured logical text, it frequently suffers from hallucination where the model prioritizes syntactic fluency over logical consistency [2].

Relying on generative output to infer reasoning processes is problematic because it conflates the model's ability to reason with its ability to articulate that reasoning in natural language [2]. By shifting the focus to internal parameter-level mechanics, the research path was restructured to prioritize the extraction of latent representations before they are subjected to the non-linear transformations of the final decoding heads. This approach adheres to the principle that internal hidden states represent a more stable, pre-output manifold of the model's learned logic.

Regarding the project objectives, the focus is placed on enhancing interpretability and maintaining computational feasibility rather than claiming a final solution to the "black-box problem". The primary aim is to investigate whether specific logical labels are decodable from internal representations, which may offer a more nuanced view of the model's latent geometry. By mapping these extracted geometries directly to nodes and edges in a symbolic graph, this project tests the hypothesis that propositional logic is organized as linearly separable features within the model's high-dimensional space.

This methodology also aligns with the research requirement for computational efficiency. By utilizing a compact 3B-parameter model and shallow classification heads, the project bypasses the need for massive, closed-source APIs or computationally expensive fine-tuning loops. This engineering path ensures that the extraction and verification architecture maintains high structural precision while operating within the memory and throughput constraints of consumer-grade hardware (such as an NVIDIA RTX 4070 Ti). This validation process establishes a reproducible, low-overhead framework capable of bridging deep learning vector spaces with verifiable symbolic graph structures.

5 Results

TBD. I hope I have enough time try single-layer and multi-layer probes, and a couple different open models to see how results change by model size.

6 Deliverables

TBD

7 References

[1] Guillaume Alain and Yoshua Bengio. Understanding intermediate layers using linear classifier probes, 2018.

[2] Fengxiang Cheng, Haoxuan Li, Fenrong Liu, Robert van Rooij, Kun Zhang, and Zhouchen Lin. Empowering llms with logical reasoning: A comprehensive survey, 2025.

[3] Zhijing Jin, Abhinav Lalwani, Tejas Vaidhya, Xiaoyu Shen, Yiwen Ding, Zhiheng Lyu, Mrinmaya Sachan, Rada Mihalcea, and Bernhard Schölkopf. Logical fallacy detection, 2022.

[4] Abhinav Lalwani, Tasha Kim, Lovish Chopra, Christopher Hahn, Zhijing Jin, and Mrinmaya Sachan. Autoformalizing natural language to first-order logic: A case study in logical fallacy detection, 2025.

[5] Xuantao Lu, Jingping Liu, Zhouhong Gu, Hanwen Tong, Chenhao Xie, Junyang Huang, Yanghua Xiao, and Wenguang Wang. Parsing natural language into propositional and first-order logic with dual reinforcement learning. In Nicoletta Calzolari, Chu-Ren Huang, Hansaem Kim, James Pustejovsky, Leo Wanner, Key-Sun Choi, Pum-Mo Ryu, Hsin-Hsi Chen, Lucia Donatelli, Heng Ji, Sadao Kurohashi, Patrizia Paggio, Nianwen Xue, Seokhwan Kim, Younggyun Hahm, Zhong He, Tony Kyungil Lee, Enrico Santus, Francis Bond, and Seung-Hoon Na, editors, *Proceedings of the 29th International Conference on Computational Linguistics*, pages 5419–5431, Gyeongju, Republic of Korea, October 2022. International Committee on Computational Linguistics.

[6] Samuel Marks and Max Tegmark. The geometry of truth: Emergent linear structure in large language model representations of true/false datasets, 2024.

[7] Jidong Tian, Yitian Li, Wenqing Chen, Liqiang Xiao, Hao He, and Yaohui Jin. Diagnosing the first-order logical reasoning ability through LogicNLI. In Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih, editors, *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 3738–3747, Online and Punta Cana, Dominican Republic, November 2021. Association for Computational Linguistics.

[8] Zhun Yang, Adam Ishay, and Joohyung Lee. Coupling large language models with logic programming for robust and general reasoning from text, 2023.